SPARK MAX Control Interfaces

The SPARK MAX can be controlled by three different interfaces, servo-style PWM, controller area network (CAN), and USB. The following sections describe the operation and protocols of these interfaces. For more details on the physical connections, see Control Connections.

PWM Interface

The SPARK MAX can accept a standard servo-style PWM signal as a control for the output duty cycle. Even though the PWM port is shared with the CAN port, SPARK MAX will automatically detect the incoming signal type and respond accordingly. For details on how to connect a PWM cable to the SPARK MAX, see CAN/PWM Port.

The SPARK MAX responds to a factory default pulse range of 1000µs to 2000µs. These pulses correspond to full-reverse and full-forward rotation, respectively, with 1500µs (±5% default input deadband) as the neutral position, i.e. no rotation. The input deadband is configurable with the REV Hardware Client or the CAN interface. The table below describes how the default pulse range maps to the output behavior.

PWM Pulse Mapping

	Output duty cycle and direction (default deadband)				
	Full Reverse	**Proportional Reverse**	**Neutral**	**Proportional Forward**	**Full Forward**
Output duty cycle, D (%)	$D = 100$	$100 < D < 0$	$D = 0$	$0 < D < 100$	$D = 100$
Input pulse width, p (µs)	$p \leq 1000$	$1000 < p < 1475$	$1475 \leq p \leq 1525$	$1525 < p < 2000$	$2000 \leq p$
Maximum pulse range, p (µs)	$500 \leq p \leq 2500$				
Valid pulse frequency, f (Hz)	$50 \leq f \leq 200$				

If a valid signal isn't received within a 60ms window, the SPARK MAX will disable the motor output and either brake or coast the motor depending on the configured Idle Mode. For details on the Idle Mode, see Idle Mode - Brake/Coast Mode.

CAN Interface

The SPARK MAX can be connected to a robot CAN network. CAN is a bi-directional communications bus that enables advanced features within the SPARK MAX. SPARK MAX must be connected to a CAN network that has the appropriate termination resistors at both endpoints. Please see the FIRST Robotics Competition Robot Rules for the CAN bus wiring requirements. Even though the CAN port is shared with the PWM port, SPARK MAX will automatically detect the incoming signal type and respond accordingly. SPARK MAX uses standard CAN frames with an extended ID (29 bits), and utilizes the FRC CAN protocol for defining the bits of the extended ID:

CAN Packet Structure

ExtID [28:24]	ExtID [23:16]	ExtID [15:10]	ExtID [9:6]	ExtID [5:0]
Device Type	Manufacturer	API Class	API Index	Device ID

Each device on the CAN bus must be assigned a unique CAN ID number. Out of the box, SPARK MAX is assigned a device ID of 0. It is highly recommended to change all SPARK MAX CAN IDs from 0 to any unused ID from 1 to 62. CAN IDs can be changed by connecting the SPARK MAX to a Windows computer and using the REV Hardware Client. For details on other SPARK MAX configuration parameters, see Configuration Parameters.
↗

Additional information about the CAN accessible features and how to access them can be found in the SPARK MAX API Information section.

Periodic Status Frames

The SPARK MAX sends data periodically back to the roboRIO. Frequently accessed data, like motor position and temperature, can be accessed using several APIs. Data is broken up into several CAN "frames" which are sent at a periodic rate. This rate can be changed manually in code, but unlike other parameters, this setting **does not persist** through a power cycle. The rate can be set anywhere from a minimum 1ms to a maximum 32767ms period. The table below describes each status frame and its available data.

Periodic Status 0 - Default Rate: 10ms

Available Data	Description
Applied **** Output	The actual value sent to the motors from the motor controller. The frame stores this value as a 16-bit signed integer, and is converted to a floating point value between -1 and 1 by the roboRIO SDK. This value is also used by any follower controllers to set their output.
Faults	Each bit represents a different fault on the controller. These fault bits clear automatically when the fault goes away.
Sticky Faults	The same as the Faults field, however the bits do not reset until a power cycle or a 'Clear Faults' command is sent.
Is Follower	A single bit that is true if the controller is configured to follow another controller.

Periodic Status 1 - Default Rate: 20ms

Available Data	Description
Motor Velocity	32-bit IEEE floating-point representation of the motor velocity in RPM using the selected sensor.
Motor Temperature	8-bit unsigned value representing: Firmware version 1.0.381 - Voltage of the temperature sensor with 0 = 0V and 255 = 3.3V. Current firmware versions - Motor temperature in °C for the NEO Brushless Motor.
Motor Voltage	12-bit fixed-point value that is converted to a floating point voltage value (in Volts) by the roboRIO SDK. This is the input voltage to the controller.
Motor Current	12-bit fixed-point value that is converted to a floating point current value (in Amps) by the roboRIO SDK. This is the raw phase current of the motor.

Periodic Status 2 - Default Rate: 20ms

Available Data	Description
Motor Position	32-bit IEEE floating-point representation of the motor position in rotations.

Periodic Status 3 - Default Rate: 50ms

Available Data	Description
Analog Sensor Voltage	10-bit fixed-point value that is converted to a floating point voltage value (in Volts) by the roboRIO SDK. This is the voltage being output by the analog sensor.
Analog Sensor Velocity	22-bit fixed-point value that is converted to a floating point voltage value (in RPM) by the roboRIO SDK. This is the velocity reported by the analog sensor.
Analog Sensor Position	32-bit IEEE floating-point representation of the velocity in RPM reported by the analog sensor.

Periodic Status 4 - Default Rate: 20ms

Available Data	Description
Alternate Encoder Velocity	32-bit IEEE floating-point representation of the velocity in RPM of the alternate encoder.
Alternate Encoder Position	32-bit IEEE floating-point representation of the position in rotations of the alternate encoder.

Periodic Status 5 - Default Rate: 200ms

Available Data	Description
Duty Cycle Absolute Encoder Position	32-bit IEEE floating-point representation of the position of the duty cycle absolute encoder.
Duty Cycle Absolute Encoder Absolute Angle	16-bit integer representation of the absolute angle of the duty cycle absolute encoder.

Periodic Status 6 - Default Rate: 200ms

Available Data	Description
Duty Cycle Absolute Encoder Velocity	32-bit IEEE floating-point representation of the velocity in RPM of the duty cycle absolute encoder.
Duty Cycle Absolute Encoder Frequency	16-bit unsigned integer representation of the frequency at which the duty cycle signal is being sent.

Use-case Examples

Position Control on the roboRIO

A user wants to implement their own PID loop on the roboRIO to hold a position. They want to run this loop at 100Hz (every 10ms), but the motor position data in Periodic Status 2 is sent at 20Hz (every 50ms).

The user can change this rate to 10ms by calling:

Pseudocode

```
setPeriodicFrameRate(PeriodicFrame.kStatus2, 10);
```

High CAN Utilization

A user has many connected CAN devices and wishes to minimize the CAN bus utilization. They do not need any telemetry feedback, and have several follower devices that are only checked for faults.

The user can set the telemetry frame rates low, and set the Periodic Status 0 frame rate low on the follower devices:

Pseudocode

```
leader.setPeriodicFrameRate(PeriodicFrame.kStatus1, 500); leader.setPeriodicFrameRate(PeriodicFrame.kStatus2, 500); follower.setPeriodicFrameRate(PeriodicFrame.kStatus0, 100); follower.setPeriodicFrameRate(PeriodicFrame.kStatus1, 500); follower.setPeriodicFrameRate(PeriodicFrame.kStatus2, 500);
```

Faster Follower Bandwidth

The user wants the follower devices to update at a faster rate: 200Hz (every 5ms).

The Periodic Status 0 frame can be increased to achieve this.

Pseudocode

```
leader.setPeriodicFrameRate(PeriodicFrame.kStatus0, 5);
```

USB Interface

The SPARK MAX can be configured and controlled through a USB connection to a computer running the REV Hardware Client. The USB interface utilizes a standard CDC (USB to Serial) driver. The command interface is similar to CAN, using the same ID and data structure, but always sends and receives a full 12-byte packet. The CAN ID is omitted (DNC) when talking directly to the device. However, the three MSB of the ID allow selection of alternate commands:

- 0b000 - Standard command - CAN ID omitted (DNC)
- 0b001 - Extended command - USB specific

All commands sent over USB receive a response. In the case that the corresponding CAN command does not receive a response, the USB interface receives an Ack command.

USB Packet Structure

ExtID [31:29]	ExtID [28:24]	ExtID [23:16]	ExtID [15:10]	ExtID [9:6]	ExtID [5:0]
USB Command Type	Device Type (2)	Manufacturer (0x15)	API Class	API Index	Device ID

USB Non-Standard Commands

Command	API Class	API Index
Enter DFU Bootloader (will also disconnect USB interface)	0	1